COMPLEMENTARY, NOT CUMULATIVE: INTERACTION EFFECTS IN PHYSICS-INFORMED NEURAL NETWORKS FOR NAVIER–STOKES VORTEX SHEDDING

Devesh Shah
Independent Researcher
deveshs@umich.edu

ABSTRACT

Physics-informed neural networks (PINNs) embed governing partial differential equations directly into the training loss, offering a promising alternative to costly CFD solvers for unsteady flows. Yet the growing list of techniques proposed to improve PINN training is typically validated one at a time, leaving open whether these techniques actually compose. We study this question in depth on the DFG/Schäfer–Turek unsteady cylinder wake benchmark. In isolation, nearly every technique performs no better than an untreated baseline. However, combining periodic (SIREN) activations with causal weighting unlocks a previously inaccessible regime, reconstructing velocity and pressure fields to within 4.1% average relative L2 error against an OpenFOAM reference solution. Adding further techniques instead causes catastrophic performance degradation, demonstrating that individually effective PINN interventions can interact nonlinearly and that more elaborate training recipes are not necessarily better.

1 INTRODUCTION

Compared to the internet-scale corpora that power today's text and vision based foundation models, most scientific domains are inherently data-starved (Willard et al., 2022). Many of the systems being modeled in science, however, are constrained by well-established physical laws. Physics-informed neural networks (PINNs) exploit this by embedding governing equations and boundary conditions directly into the training loss, approximating solutions to partial differential equations (PDEs) with only a fraction of the data such problems would otherwise require (Raissi et al., 2019).

Since this formulation was introduced, PINNs have been applied across a wide range of domains, including solid mechanics (Haghighat et al., 2021), heat transfer (Cai et al., 2021), and biomedical flow modeling (Kissas et al., 2020). However, these models face well-documented challenges when scaling to more complex problems, largely due to training instability when the loss depends on higher-order derivatives (Krishnapriyan et al., 2021; Wang et al., 2021).

Fluid dynamics is among the most consequential of these complex problem classes: predicting fluid behavior underpins nearly every discipline of physical engineering. Because traditional computational fluid dynamics (CFD) solvers become especially costly at higher Reynolds numbers (Kochkov et al., 2021), there is strong motivation for faster learned surrogates such as PINNs. Yet this promise is not automatic: prior work has documented failures even for standard setups (Chuang & Barba, 2023). As PINN research has grown, so has the number of proposed fixes, typically validated in isolation against different baselines. We systematically test whether these techniques compose on the standard DFG/Schäfer–Turek unsteady cylinder wake benchmark. Our key contributions are:

1. A systematic evaluation of widely-used PINN techniques spanning architectural changes, activation functions, and loss reweighting schemes.
2. Identification of the specific combination (periodic activations paired with causal weighting) needed to reliably learn vortex shedding.
3. An analysis showing why further additions to this pairing, such as fourier features or self-adaptive weighting, each independently break it down.

Code for all experiments is publicly available at `https://github.com/deveshshah1/Navier_Stokes_Exploration_with_PINNs.git`.

2 Background & Related Work

The Navier–Stokes equations rank among the most notoriously difficult systems in mathematical physics: proving that smooth solutions always exist in three dimensions is one of the seven unsolved Millennium Prize Problems (Carlson et al., 2006). These equations describe how a fluid's velocity field evolves under the forces acting on it:

$$\nabla \cdot \mathbf{u} = 0 \tag{1a}$$

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu \nabla^2 \mathbf{u} \tag{1b}$$

where $\mathbf{u} = (u, v)$ is the velocity field, p is pressure, ρ is fluid density, and ν is kinematic viscosity. Two properties make this system hard to solve directly: the convective term $(\mathbf{u} \cdot \nabla)\mathbf{u}$ is nonlinear, ruling out closed-form solutions in general, and pressure has no evolution equation of its own, so it must be solved for indirectly at every step to keep the velocity field divergence-free. This coupling is why conventional CFD solves a discretized version of these equations on a spatial mesh, advancing the solution forward in time while iteratively coupling the pressure and velocity fields to enforce incompressibility (Patankar, 2018). The Reynolds number, the ratio of convective to viscous forces, governs this cost: larger values push flows into unsteady, vortex-dominated regimes (e.g. the unsteady loading on turbine blades) that demand finer meshes and smaller time steps.

PINNs have famously struggled as target dynamics grow more complex. Krishnapriyan et al. (2021) showed that this failure is not due to insufficient network capacity: the same architecture that fails as a PINN can represent the correct solution when trained in a purely supervised fashion. Instead, embedding the PDE residual into the loss creates an optimization landscape that becomes extremely difficult to navigate. Wang et al. (2021) identified a related but distinct failure mode: gradients backpropagated from different loss terms (physics, boundary, and initial conditions) can become severely imbalanced during training. Similarly, Rahaman et al. (2019) identified a spectral bias in standard MLPs, the baseline PINN architecture: a tendency to preferentially learn low-frequency functions. This bias was later shown to also limit coordinate-based neural representations such as images and 3D scenes (Tancik et al., 2020).

Each of these diagnoses led to a wave of new techniques: loss-side fixes such as causal (Wang et al., 2022) and self-adaptive weighting (McClenny & Braga-Neto, 2020), and architectural fixes such as Fourier feature encoding (Tancik et al., 2020) and SIREN activations (Sitzmann et al., 2020). These are explored and explained in more detail throughout this paper.

Vortex shedding around a cylinder, the specific problem studied in this work, has also drawn dedicated PINN research in its own right - although published work has largely approached the problem through specialized, one-off loss reformulations or architectural changes. Wei et al. (2026) reformulate the loss function itself, adding velocity and pressure correction terms inspired by the classical SIMPLE pressure-velocity coupling algorithm to capture the evolution of vortex shedding. Jiang & Chen (2025) replace the standard MLP with a U-Net++, predicting a stream function rather than velocity and pressure directly, and approximating derivatives with gradient-free convolutional filters rather than automatic differentiation. Chuang & Barba (2023) take a more diagnostic approach, using the standard PINN formulation to show that it cannot sustain vortex shedding without continuous data supervision, tracing this failure to numerically dissipative dynamic modes. Our work complements this literature by holding the standard PINN formulation fixed and systematically testing how established, general-purpose techniques combine, rather than proposing a new specialized architecture or loss for this problem.

Beyond PINNs specifically, operator-learning methods such as DeepONet (Lu et al., 2021) and the Fourier Neural Operator (Li et al., 2020) offer an alternative paradigm for fast CFD surrogates, directly approximating the solution operator of a PDE rather than embedding its residual into the loss. However, unlike our setting, these approaches typically require substantial training data spanning many solved instances; thus, we do not explore these methods further in this work.

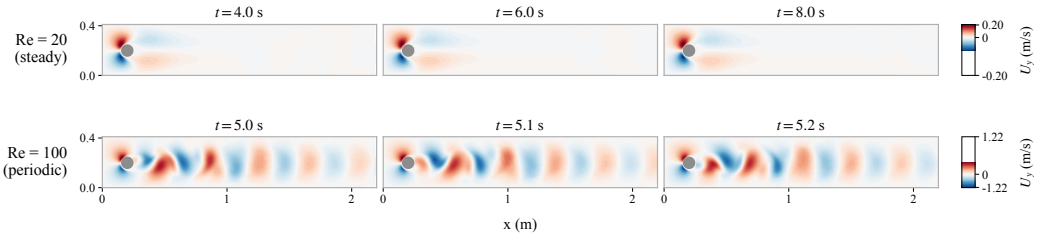


Figure 1: U_y velocity field snapshots from our OpenFOAM ground-truth dataset. Top: $Re = 20$, steady laminar wake, essentially unchanged across a 4-second span. Bottom: $Re = 100$, periodic Kármán vortex street, visibly evolving within a fraction of one shedding period.

3 DATASET

We study the two-dimensional flow of an incompressible Newtonian fluid past a circular cylinder confined within a rectangular channel, following the widely adopted Schäfer–Turek (ST) benchmark (Schäfer et al., 1996). This benchmark provides well-established reference values for drag and lift coefficients and Strouhal number, making it an ideal testbed for validating PINN approaches to incompressible flow. All simulations use a shared fluid and geometry; only the inlet velocity, and consequently the Reynolds number, differs between cases. We only consider the steady-inlet (ST 2D-1) variant of the benchmark, governed by the Navier–Stokes equations given in equation 1.

We simulate two Reynolds numbers spanning qualitatively different flow physics, allowing us to quantify how solution complexity affects PINN accuracy. At $Re = 20$, the flow is laminar and settles into a steady, symmetric wake. Any temporal variation the PINN predicts after convergence reflects error rather than physics, making this a controlled test of the steady-state limit. At $Re = 100$ the flow is unsteady and sheds a periodic Kármán vortex street with a well-defined Strouhal number near 0.30, requiring the network to learn the spatial structure of the shed vortices and their periodic timing. Figure 1 visualizes this contrast directly.

All reference flow fields are generated using OpenFOAM v2512, an open-source finite-volume CFD framework. The mesh is first generated in Gmsh before being imported into OpenFOAM. Both cases are solved with the transient laminar solver `icoFoam` and PISO pressure-velocity coupling. The mesh is a graded, quad-dominant extrusion with 50,184 cells, refined near the cylinder wall and through the wake. For $Re = 20$ we simulate $t \in [0, 10]$s at $\Delta t = 0.001$s; for $Re = 100$ we simulate $t \in [0, 8]$s, the duration specified by the benchmark, at $\Delta t = 0.0002$s. Both cases write the full flow field every 0.1s, yielding 100 and 80 snapshots respectively. Full details appear in Appendix A.

Before using these outputs as PINN training and evaluation data, we validate them against the published ST benchmark reference values. At $Re = 20$, our converged drag coefficient matches the benchmark to within 0.2%. At $Re = 100$, our time-averaged drag matches to within 7.8%, while lift amplitude and Strouhal number match to within 1%. The larger drag discrepancy at $Re = 100$ reflects the known sensitivity of integrated drag to near-wall mesh resolution at higher Reynolds number, and caps how closely a PINN trained on these fields can match the true ST solution.

For each simulation, we extract cell-centre velocity, pressure, and coordinate fields from the OpenFOAM output at every written time step. The combined dataset totals approximately 9 million spatiotemporal observations across both Reynolds numbers.

4 METHODS

4.1 BASE PINN FORMULATION

We validate the base PINN formulation on the Re=20 case before applying it to the harder Re=100 dynamics studied in the rest of this paper. At Re=20, the flow is steady, so the model only needs to learn a time-invariant, symmetric wake. Training uses no direct data supervision, relying only on governing physics and boundary/initial conditions. At steady state ($t \geq 2.0$s), relative L2 errors against OpenFOAM are all within 3% (Table 1), and the predicted pressure drop across the cylinder

matches to within 1%. This confirms the configuration below is correctly implemented, and this exact recipe is carried over unchanged to Re=100 for the remainder of the paper's ablations.

Architecture. A fully-connected multilayer perceptron maps the spatiotemporal coordinates (x, y, t), normalized to $[-1, 1]$ per coordinate using the domain bounds, to the flow variables (u, v, p). The network has 8 hidden layers of width 128, $\tanh$ activations, Xavier-uniform weight initialization, and zero-initialized biases.

Loss formulation. The training loss is a weighted sum of three terms:

$$\mathcal{L} = \lambda_{phys}\mathcal{L}_{phys} + \lambda_{bc}\mathcal{L}_{bc} + \lambda_{ic}\mathcal{L}_{ic}, \tag{2}$$

with baseline weights $\lambda_{phys} = 3.0$, $\lambda_{bc} = 5.0$, $\lambda_{ic} = 0.2$ and losses defined as:

$\mathcal{L}_{phys}$ The mean squared residual of the continuity and momentum equations (equation 1a, equation 1b), evaluated via automatic differentiation. The momentum residual requires the kinematic viscosity ν, set at $\nu = 0.001$ m^2/s for the Re=20 case, consistent with the mean velocity and cylinder diameter in equation 8.

$\mathcal{L}_{bc}$ Sums three boundary terms: the inlet, matched against the analytic Schäfer–Turek parabolic profile given in equation 9; the outlet, held at zero pressure; and no-slip, zero velocity on the channel walls and cylinder surface.

$\mathcal{L}_{ic}$ Enforces the fluid starting at rest ($u = v = 0$ at $t = 0$).

Sampling. Collocation, boundary, and initial-condition points are drawn i.i.d. uniformly at random and resampled fresh at every training step: 8000 collocation points, 500 points each at the inlet and outlet, 500 points on each of the three no-slip surfaces, and 300 initial-condition points.

Training. We train with AdamW, learning rate 10^{-3}, cosine-annealed over 125K steps, and gradient norm clipped to 1.0.

4.2 Candidate Techniques

Here we introduce a variety of techniques proposed in the PINN literature, each addressing a specific, previously-documented failure mode of physics-only training (Section 2). Each can be added to the base formulation of Section 4.1 independently of the others.

4.2.1 Fourier Feature Encoding

Standard MLPs with smooth activations (e.g. tanh) struggle to learn high-frequency signals: left to gradient descent, they preferentially settle into the smoothest function that satisfies the loss well, a phenomenon known as spectral bias (Rahaman et al., 2019). This is a common obstacle in PINN solutions, and especially relevant to ours: vortex shedding is periodic, and with a shedding period of 0.336s within our 8s training window, it creates a genuinely high-frequency signal in time.

Fourier feature encoding counters this by lifting the raw input coordinates into a high-frequency basis before they reach the network (Tancik et al., 2020). Each coordinate group is mapped through

$$\gamma(\mathbf{x}) = [\sin(2\pi B\mathbf{x}),\ \cos(2\pi B\mathbf{x})] \in \mathbb{R}^{2m}, \tag{3}$$

where the frequency matrix B is sampled once from $\mathcal{N}(0, \sigma^2)$ and then held fixed as a non-trainable buffer for the rest of training. We implement this as two separate encoders, one each for the spatial and time axes, with $\sigma_{spatial} = 2.0$ and $\sigma_{time} = 1.0$.

Table 1: Relative L2 error against OpenFOAM for all baseline experiments. *Note Re=20 results are reported at steady state ($t \geq 2.0$s); all other rows report error over the full trajectory.

Re	Physics Loss	Data Supervision	Ux	Uy	p
20	✓	×	1.1%*	2.1%*	2.8%*
100	✓	×	68.3%	86.2%	118.4%
100	✓	✓	19.8%	70.5%	36.1%
100	×	✓	21.6%	108.3%	35.5%

4.2.2 CAUSAL WEIGHTING

For time-dependent PDEs, the true solution is causal: the state at time t depends only on the initial condition and the dynamics that unfolded before t, not on the solution at later times. Standard PINN training ignores this structure entirely, since collocation points are drawn across the whole time domain. This lets the optimizer reduce the loss by fitting late-time residuals before resolving earlier dynamics, even though a low late-time residual is only physically meaningful if those early dynamics were learned correctly first. Wang et al. (2022) argue this is a major source of PINN training failure, and propose weighting the physics loss so that later times only contribute once earlier times are already well fit.

We implement a discretized, batch-local approximation of this idea: each batch's collocation points are binned into $M = 20$ equal intervals over $[t_{min}, t_{max}]$, and the mean physics residual within each bin, $\bar{r}_k$, is computed. Bin weights are then set by the cumulative residual in all earlier bins,

$$w_k = \exp\left(-\epsilon \sum_{j<k} \bar{r}_j\right), \tag{4}$$

so that a bin only receives substantial weight once the physics residual in every preceding bin has been driven down. Each collocation point inherits the weight of its time bin, and the physics loss becomes the weighted mean $\frac{1}{N}\sum_i w_i r_i$. We set $\epsilon = 1.0$.

4.2.3 HARD BOUNDARY CONDITION ENFORCEMENT

Boundary conditions in a standard PINN are enforced softly, as a term in the loss that the network is only encouraged, not guaranteed, to satisfy. This creates competition with the other loss terms, so the boundary condition can be violated where its weight is outmatched by conflicting physics or data gradients. Sukumar & Srivastava (2022) propose constructing the network's output so that boundary conditions are satisfied exactly, removing the constraint from the optimization problem.

The no-slip condition on the cylinder surface is a natural candidate for this treatment: it is where the steepest velocity gradients occur, so any imperfection here has outsized downstream effects. We construct a smooth distance-based mask around the cylinder,

$$d(x, y) = \tanh\left(10 \cdot \frac{r_{dist} - r}{r}\right), \qquad r_{dist} = \sqrt{(x - c_x)^2 + (y - c_y)^2}, \tag{5}$$

and multiply it into the raw network output for the velocity components, $u = d(x, y)\, u_{raw}$ and $v = d(x, y)\, v_{raw}$. Thus, on the cylinder surface, d vanishes regardless of what the network predicts, enforcing no-slip architecturally rather than through the loss. Away from the cylinder, $d \to 1$, so the mask leaves the solution unaffected outside a thin band near the wall.

4.2.4 PERIODIC (SIREN) ACTIVATIONS

Sinusoidal activations offer a complementary way to address the spectral bias described in Section 4.2.1, changing the activation function itself rather than the input encoding. Sitzmann et al. (2020) propose replacing $\tanh$ or ReLU with $\sin(\omega z)$ throughout the network, so that every layer, not just the input, is built from oscillatory functions capable of representing fine detail. Since the physics loss requires differentiating the network's output multiple times in the momentum equation, this matters directly for us: repeated differentiation of a sinusoid remains a bounded, well-behaved sinusoid, whereas the derivatives of $\tanh$ degrade and saturate.

Following Sitzmann et al. (2020), we set ω of the first layer to 30 and 1 for all subsequent layers. SIREN also requires a different initialization scheme: first-layer weights are drawn from $\mathcal{U}(-1/n_{in}, 1/n_{in})$, all later layers from $\mathcal{U}(-\sqrt{6/n_{in}}, \sqrt{6/n_{in}})$, with biases zero-initialized.

4.2.5 SELF-ADAPTIVE LOSS WEIGHTS

Balancing multiple loss terms against each other is a difficult hyperparameter optimization problem. McClenny & Braga-Neto (2020) address this by making each weight a learnable parameter and framing the loss as a min-max game: the network minimizes the loss with respect to its weights,

while the weights themselves are simultaneously updated to maximize it. This gives training a self-correcting mechanism for loss balancing that responds to how training is actually progressing, rather than a single fixed weighting decided in advance.

We reparameterize each weight in log-space, $\lambda_k = \exp(\ell_k)$, guaranteeing $\lambda_k > 0$ for any value of ℓ_k. The network weights θ and the log-weights ℓ are optimized separately: θ is updated by ordinary gradient descent on the loss L, while ℓ is updated by gradient ascent (learning rate 10^{-3}) on the same loss:

$$\ell_k \leftarrow \ell_k + \text{lr} \cdot \nabla_{\ell_k} L = \ell_k + \text{lr} \cdot \lambda_k L_k, \tag{6}$$

Since $\nabla_{\ell_k} L = \lambda_k L_k$, this pushes λ_k up fastest for whichever component L_k currently has the largest loss. After each update, ℓ_k is clamped to $[-3, 5]$ to keep the ascent dynamics from diverging.

4.2.6 POST-HOC SECOND-ORDER FINE-TUNING (L-BFGS)

Adam is a first-order optimizer: it adapts its step size using only the gradient's own recent history, with no information about the curvature of the loss surface around it. This makes it effective at making rapid progress across a rough, high-dimensional loss landscape early in training, but it often fails to converge tightly once training is already close to a local optimum. A common practice in PINN training is to follow Adam with a quasi-Newton method, most often L-BFGS, which builds an approximation of the inverse Hessian from recent gradient history to take curvature-aware steps and refine a solution beyond what Adam alone reaches (Raissi et al., 2019).

We fine-tune from our best Adam checkpoint using an initial step size of 1.0, rescaled at each step by a strong-Wolfe line search, and an inverse-Hessian approximation built from a history of 100 past gradient-step pairs. Unlike the continuous resampling used during Adam training, the collocation, boundary, initial-condition, and data points are held fixed for each L-BFGS step, so that the loss and gradient stay consistent across the multiple re-evaluations required by the line search.

4.3 EXPERIMENTAL DESIGN

Applying the base formulation from Section 4.1 to Re=100 fails badly (Table 1): relative L2 errors reach 68% (Ux), 86% (Uy), and 118% (p). This is not a noisy version of the correct dynamics, but a collapse to a smoothed, quasi-steady near-field response around the cylinder. The model effectively finds it easier to zero out the temporal derivative than to resolve the true periodic dynamics.

To address this, we test the full set of candidate techniques from Section 4.2 individually, but find that none meaningfully move the model out of this collapse. We therefore introduce an additional loss term, $\mathcal{L}_{data}$, that grounds training with direct supervision from a held-out 1% (40,000-point) sample of our OpenFOAM ground truth. Adding this term partially rescues the baseline on Ux and p, but Uy, the velocity component most directly driven by shedding, barely moves (Table 1).

To verify the physics loss itself contributes, we test a data-only control with the physics loss removed. It does not outperform the physics-only baseline (Table 1), but fails differently: it produces roughly correct oscillation amplitude decorrelated from the true shedding phase, rather than the near-zero variance of the physics-only runs. This confirms physics and data address different parts of the failure, and neither alone suffices. We adopt the physics-constrained, data-supervised configuration as the baseline for all results below; consistent with Chuang & Barba (2023), some form of data supervision proves necessary for learning this problem at all.

Given a limited compute budget of T4 GPUs, the base formulation in Section 4.1 is itself intentionally kept at a reduced scale. None of our runs are trained to full convergence and loss continues to decrease slowly well beyond our training budget. We rely on the assumption that relative comparisons between configurations remain informative under this shared, fixed compute budget.

With this data-supervised configuration as our baseline, we evaluate the six candidate techniques from Section 4.2 in two stages. We first test each technique individually to measure how much of the shedding failure mode any single technique can address on its own. We then use these individual results to guide a series of combination searches, pairing and stacking techniques to see which combinations succeed where the individual techniques fall short, and which instead degrade performance relative to their parts.

Table 2: Relative L2 error, predicted Strouhal number, wake-probe amplitude ratio, and wake-probe correlation (each averaged over $x/D = 2, 4, 6$) for various experiments. True St=0.300 for all rows.

Configuration	Ux	Uy	p	St pred	Amp. ratio	Corr.
Baseline	19.8%	70.5%	36.1%	0.013	0.005	0.005
Individual techniques						
Fourier Features	19.4%	70.5%	35.2%	0.025	0.007	0.003
Causal Weighting	19.2%	70.5%	35.4%	0.013	0.007	0.005
SIREN Activation	19.5%	70.5%	35.3%	0.025	0.010	0.007
Hard Boundary Conditions	22.2%	72.4%	45.0%	0.025	0.005	−0.002
Self-Adaptive Weights	36.1%	89.8%	91.6%	0.013	0.009	−0.005
Combinations						
Fourier Features + Causal Weighting	11.7%	38.4%	21.9%	**0.300**	0.720	0.861
Fourier Features + Causal Weighting (2% data)	12.1%	38.1%	22.2%	**0.300**	0.755	0.865
SIREN + Causal Weighting	**7.8%**	**17.1%**	**16.7%**	**0.300**	0.833	**0.988**
SIREN + Causal Weighting ($\epsilon = 5$)	9.6%	24.5%	19.0%	**0.300**	**0.892**	0.964
SIREN + Causal Weighting + Self-Adaptive	21.8%	75.7%	37.9%	0.088	0.110	0.005
Fourier Features + SIREN + Causal Weighting	58.1%	99.8%	98.1%	0.025	0.002	0.053

4.4 Evaluation Metrics

To avoid a single aggregate error hiding the qualitatively different failure modes discussed in Section 4.3, we report three tiers of metrics. Our primary accuracy measure is the relative L2 error, computed per field over the full spatiotemporal domain. To separate models that fail to oscillate at all from those that oscillate with the wrong phase, we additionally report temporal correlation and oscillation amplitude ratio at fixed downstream wake probes. Finally, we report two domain-standard quantities from the CFD literature, pressure drop across the cylinder and Strouhal number.

5 Results

Section 4.3 established a working baseline. We now turn to this paper's central questions: how much each individual technique matters, which combination reliably learns the shedding dynamics, and, for the combinations that fail, what specific mechanism is responsible.

Table 2 summarizes every individual technique and combination tested. Individually, none of the five techniques recovers any coherent shedding tone or meaningful wake oscillation: L2 error, pressure, and dominant frequency all barely move from baseline. Pairing causal weighting with either spectral-bias fix, SIREN or Fourier Features, changes this qualitatively: both recover the true Strouhal number and produce strong phase-locked correlation at every wake probe, with SIREN the stronger pairing on every metric. Stacking further onto this pairing is where the interesting failures live: tightening causal weighting or doubling data supervision leaves headline error roughly unchanged while modestly improving temporal fidelity, adding Self-Adaptive Weights or Fourier Features on top of a working recipe both actively destroy it: one an optimization failure, the other an architectural one, each traced to a specific mechanism below.

Table 3 tests whether SIREN+causal weighting's remaining error is a capacity or training-budget limitation, given the reduced-scale setup of Section 4.3: training our current architecture for longer (4000 epochs), training a larger model (6 hidden layers of width 256), and fine-tuning the converged larger model with L-BFGS. Both training longer and using a larger model help substantially, more than halving error, while L-BFGS destroys nearly all of these gains.

Figure 2 shows the impact of these configurations directly. The baseline never learns the oscillating pressure-drop behavior at all, SIREN+causal weighting recovers it but at reduced amplitude, and the deeper, fully-converged model is nearly indistinguishable from OpenFOAM, including the wake-deficit double-hump structure in the velocity profiles. Fourier Features stacked on SIREN+causal weighting instead collapses entirely, a failure mode we return to in Section 5.2.

Table 3: Understanding capacity and training budget limitations of the best configuration.

Configuration	Ux	Uy	p	St pred	Amp. ratio	Corr.
SIREN + Causal Weighting (reference)	7.8%	17.1%	16.7%	**0.300**	0.833	0.988
+ more epochs, trained to convergence	4.4%	10.8%	10.6%	**0.300**	0.901	0.996
+ deeper model, trained to convergence	**3.0%**	**4.8%**	**4.6%**	**0.300**	**0.983**	**1.000**
+ L-BFGS fine-tuned (on deeper model)	22.4%	80.3%	43.0%	0.013	0.041	0.003

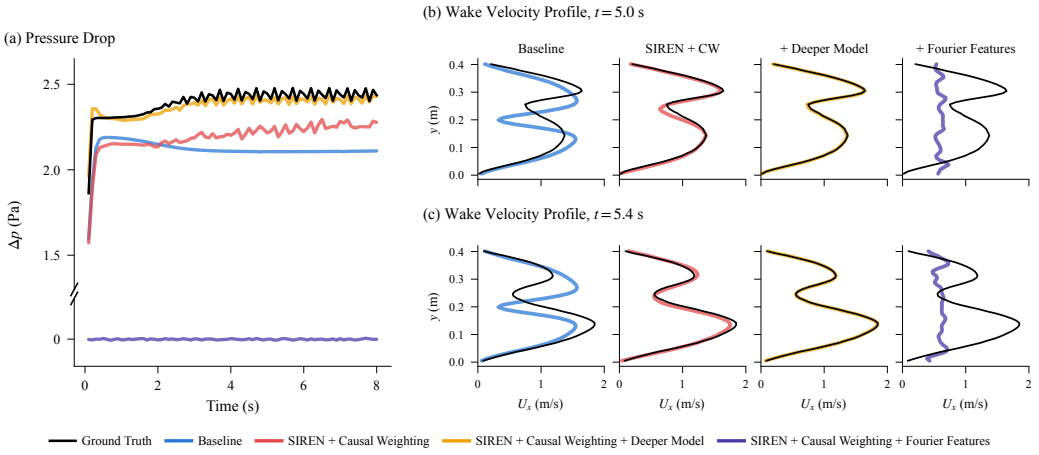


Figure 2: For various configurations we compare (a) Pressure drop $\Delta p(t)$ and (b, c) streamwise wake velocity profiles $U_x(y)$ at $t = 5.0$s and $t = 5.4$s against the ground truth.

5.1 Optimization Instability

We now examine why these failures occur, starting with a pattern visible directly in the training checkpoints: across many of the experiments, especially the individual techniques tested without pairing, each catastrophic run's best checkpoint was captured within the first 50 epochs of a 2000-epoch schedule. This is a real taxonomy: optimization-dynamics failures break training itself, early and permanently, distinct from configurations that train normally but simply converge to a worse solution.

We take a closer look at one technique in particular to understand why training freezes so early: Self-Adaptive Weights. Figure 3(a) shows the training loss spiking early once Self-Adaptive Weights are introduced. Panel (b) shows why: the four self-adaptive weights escalate toward their clamp ceiling (e^5, equation 6) in a staggered order, with λ_{phys} last. The adversarial ascent inflates whichever term is currently easiest to increase loss on first, boundary and data fitting, well before it reaches the physics residual. By the time every weight has been driven to its bound, the optimization has already been destabilized for the better part of one hundred epochs. This staggered blowup, not a permanently starved physics term, is what produces the early freeze in the best-checkpoint epoch.

L-BFGS's collapse, on top of the converged deeper model, shares this same early-freeze signature but for a distinct mechanism. L-BFGS did not drift or diverge gradually: it took one large step that flattened and redistributed the weights, erasing the sharp, high-magnitude weights SIREN relies on to represent high-frequency content, and its line search then stalled completely. The failure is a single catastrophic update, not a gradual overshoot.

5.2 Architectural Incompatibility

The collapse of Fourier Features + SIREN + Causal Weighting has a different cause entirely. We compare per-term training loss at initialization (epoch 0) for SIREN+Causal Weighting with and without Fourier Features. Adding Fourier Features inflates the physics residual by $79\times$ relative to SIREN alone, while the other three terms ($\mathcal{L}_{bc}$, $\mathcal{L}_{ic}$, $\mathcal{L}_{data}$) stay within the same order of magnitude.

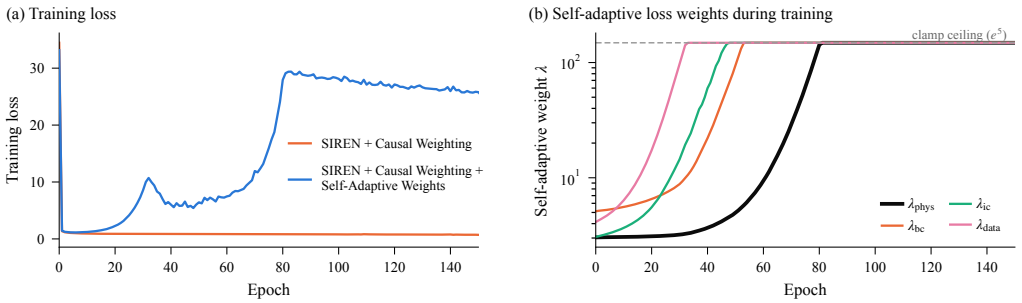


Figure 3: (a) Training loss for SIREN+causal weighting with and without Self-Adaptive Weights. (b) Self-adaptive loss weights over training: all four escalate to the clamp ceiling in a staggered order, data first, physics last.

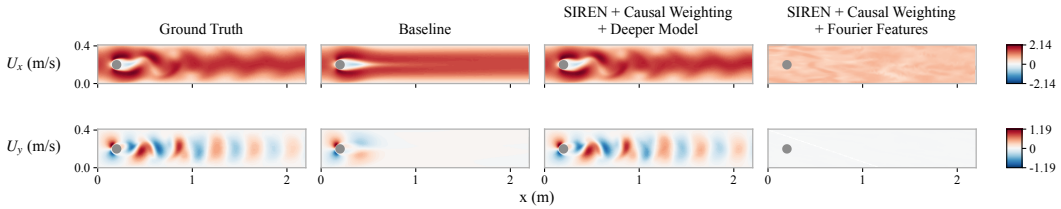


Figure 4: Field comparison at $t = 5.0$s for various experimental configurations. Note that SIREN + Fourier Features together collapse to a near-uniform field.

This imbalance is fatal: by epoch 15, the physics loss has collapsed to approximately 4×10^{-6}, essentially zero, and stays pinned there for the remaining $\sim$1300 epochs of the run.

The physics residual (equation 1b) is dominated by derivative terms, $u_x, u_y, u_t, u_{xx}, u_{yy}$, v_x, v_y, v_{xx}, v_{yy}, all of which vanish identically if the network's output is constant. Driving physics loss to zero therefore does not require learning the flow field: collapsing the output to a near-constant function trivially satisfies it. Comparing the trained checkpoints' output-layer weights confirms this: the final linear layer shrinks by roughly $21\times$ when Fourier Features is added (output layer norm shrinks from 1.397 to 0.066). This is the direct cause of the field collapse visible in Figure 4, where predicted U_y and pressure both flatten to a narrow band near zero across the whole domain.

Why does stacking Fourier Features onto SIREN specifically cause this, rather than simply adding capacity? The two techniques compound multiplicatively rather than additively at the first layer. A Fourier feature is $\gamma(x) = \sin(2\pi\beta x)$ with $\beta \sim \mathcal{N}(0, \sigma^2)$; feeding it into a SIREN first layer, $h(x) = \sin(\omega_0(w \cdot \gamma(x) + b))$, gives by the chain rule

$$\frac{dh}{dx} = \omega_0 \cos(\cdot) \cdot w \cdot \underbrace{2\pi\beta \cos(2\pi\beta x)}_{d\gamma/dx}, \tag{7}$$

so the effective first-layer frequency is $\omega_0 \cdot 2\pi\beta$ rather than just ω_0. With our hyperparameters ($\omega_0 = 30$, $\sigma_{spatial} = 2.0$), this is roughly a $2\pi\sigma \approx 12.6\times$ amplification, and because the viscous term needs a second derivative, this compounds further. Measuring PDE-residual derivatives at random initialization confirms this structurally: the second-derivative standard deviation is roughly $2.4\times$ larger with Fourier Features added. As a result, compounding frequencies inflate the initial physics residual by roughly $79\times$, and gradient descent takes the cheapest available exit by collapsing the output layer toward a degenerate near-constant solution where every derivative term vanishes trivially.

6 CONCLUSION

We presented a systematic ablation study of five widely-used PINN techniques on the DFG/Schäfer–Turek unsteady cylinder wake benchmark, evaluating each individually, in combination, and stacked beyond their best-performing pairing. This paper's central finding is that these techniques are complementary, not cumulative: which pairing works matters far more than how many techniques are stacked together.

An open question is whether this complementarity generalizes beyond our controlled benchmark, to three-dimensional, higher-Reynolds-number flows where the underlying dynamics are far more complex, and where techniques validated here may compose differently, or not at all. More broadly, we hope the diagnostic approach taken here, tracing failures to specific, checkable mechanisms in training checkpoints and network weights rather than treating them as unexplained instability, proves useful for evaluating other technique combinations beyond the ones studied here.

REFERENCES

Shengze Cai, Zhicheng Wang, Sifan Wang, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks for heat transfer problems. *Journal of Heat Transfer*, 143(6):060801, 2021.

James Carlson, Arthur Jaffe, and Andrew Wiles (eds.). *The Millennium Prize Problems*. American Mathematical Society, Providence, RI, 2006. ISBN 978-0-8218-3679-8.

Pi-Yueh Chuang and Lorena A Barba. Predictive limitations of physics-informed neural networks in vortex shedding. *arXiv preprint arXiv:2306.00230*, 2023.

Ehsan Haghighat, Maziar Raissi, Adrian Moure, Hector Gomez, and Ruben Juanes. A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics. *Computer Methods in Applied Mechanics and Engineering*, 379:113741, 2021.

Chunhao Jiang and Nian-Zhong Chen. Gradient-free physics-informed neural networks (gf-pinns) for vortex shedding prediction in flow past square cylinders. *Computers in Industry*, 169:104304, 2025.

Georgios Kissas, Yibo Yang, Eileen Hwuang, Walter R Witschey, John A Detre, and Paris Perdikaris. Machine learning in cardiovascular flows modeling: Predicting arterial blood pressure from non-invasive 4d flow mri data using physics-informed neural networks. *Computer methods in applied mechanics and engineering*, 358:112623, 2020.

Dmitrii Kochkov, Jamie A Smith, Ayya Alieva, Qing Wang, Michael P Brenner, and Stephan Hoyer. Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21):e2101784118, 2021.

Aditi Krishnapriyan, Amir Gholami, Shandian Zhe, Robert Kirby, and Michael Mahoney. Characterizing possible failure modes in physics-informed neural networks. *Advances in neural information processing systems*, 34:26548–26560, 2021.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.

Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021.

Levi McClenny and Ulisses Braga-Neto. Self-adaptive physics-informed neural networks using a soft attention mechanism. *arXiv preprint arXiv:2009.04544*, 2020.

Suhas Patankar. *Numerical heat transfer and fluid flow*. CRC press, 2018.

Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International conference on machine learning*, pp. 5301–5310. PMLR, 2019.

Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

Michael Schäfer, Stefan Turek, Franz Durst, Egon Krause, and Rolf Rannacher. Benchmark computations of laminar flow around a cylinder. In *Flow simulation with high-performance computers II: DFG priority research programme results 1993–1995*, pp. 547–566. Springer, 1996.

Vincent Sitzmann, Julien Martel, Alexander Bergman, David Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. *Advances in neural information processing systems*, 33:7462–7473, 2020.

Natarajan Sukumar and Ankit Srivastava. Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *Computer Methods in Applied Mechanics and Engineering*, 389:114333, 2022.

Matthew Tancik, Pratul Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in neural information processing systems*, 33:7537–7547, 2020.

Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021.

Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality is all you need for training physics-informed neural networks. *arXiv preprint arXiv:2203.07404*, 2022.

Chang Wei, Yuchen Fan, Chin Chun Ooi, Jian Cheng Wong, Heyang Wang, and Pao-Hsiung Chiu. Bridging computational fluid dynamics algorithm and physics-informed learning: Simple-pinn for incompressible navier-stokes equations. *arXiv preprint arXiv:2603.24013*, 2026.

Jared Willard, Xiaowei Jia, Shaoming Xu, Michael Steinbach, and Vipin Kumar. Integrating scientific knowledge with machine learning for engineering and environmental systems. *ACM Computing Surveys*, 55(4):1–37, 2022.

A CFD: OpenFOAM Configuration

This appendix documents the full OpenFOAM configuration used to generate the reference dataset in this work: the problem geometry and governing relations, mesh generation, boundary conditions, solver configuration, and the specific parameters used for each of the two Reynolds number cases.

A.1 Problem Geometry and Governing Relations

We simulate two-dimensional, incompressible flow of a Newtonian fluid past a circular cylinder confined within a rectangular channel, following the Schäfer–Turek (ST) benchmark (Schäfer et al., 1996). The domain is a channel of length L and height H, with a cylinder of diameter D positioned off-centre near the inlet (Table 4). The flow is governed by the incompressible Navier–Stokes equations given in equation 1, where $\mathbf{u} = (u, v)$ is the velocity field, p is pressure, ρ is fluid density, and ν is kinematic viscosity. The Reynolds number is defined as

$$Re = \frac{U_{mean} D}{\nu}, \tag{8}$$

where U_{mean} is the mean inlet velocity; it is the only quantity that differs between our two simulated cases (Sections A.5 and A.6). The inlet boundary follows the parabolic (Hagen–Poiseuille) profile specified by the ST benchmark,

$$u(0, y, t) = \frac{6 U_{mean} y (H - y)}{H^2} \tag{9}$$

$$v(0, y, t) = 0, \tag{10}$$

and all simulations are initialised from rest ($u = v = 0$ everywhere at $t = 0$). Drag and lift coefficients, used throughout this appendix to validate our simulations against the published ST reference values, are computed from OpenFOAM's `forceCoeffs` function object using reference velocity $U_{ref} = U_{mean}$, reference length $L_{ref} = D$, and reference area $A_{ref} = D \times$ depth:

$$C_d = \frac{2 F_d}{\rho U_{mean}^2 D \,\text{depth}} \tag{11}$$

$$C_l = \frac{2 F_l}{\rho U_{mean}^2 D \,\text{depth}} \tag{12}$$

Table 4: Computational geometry. All dimensions are in metres.

Parameter	Value	Units
Channel length L	2.2	m
Channel height H	0.41	m
Cylinder centre (c_x, c_y)	(0.20, 0.20)	m
Cylinder radius R	0.05	m
Cylinder diameter D	0.10	m
Domain extent (z)	0.01	m (pseudo-2D)

A.2 Mesh Generation and Post-Processing

The mesh is generated in Gmsh as a two-dimensional quad-dominant surface mesh with a single-layer hexahedral extrusion in z, giving a fully structured pseudo-2D domain, and is imported into OpenFOAM with `gmshToFoam`. Graded refinement toward the cylinder wall and through the wake is achieved with Gmsh distance and threshold fields applied to the cylinder arc curves (Table 5). The front and back faces of the domain are treated as `empty` (pseudo-2D) patches.

For each written snapshot, velocity components u, v, and kinematic pressure p/ρ are read from the OpenFOAM binary-format field files; only internal cell values are extracted, and boundary face values are discarded. Cell-centre coordinates are obtained from the `C` field written by `writeCellCentres`, and the z-coordinate is discarded since the domain is pseudo-2D.

Table 5: Mesh generation parameters. The mesh is shared by both $Re = 20$ and $Re = 100$ simulations.

Parameter	Value	Units / notes
Meshing tool	Gmsh 4.x	—
Element type	Hexahedral (quad-extruded)	—
Total cells	50,184	—
Far-field element size $l_{c,far}$	0.030	m
Wake refinement size $l_{c,wake}$	0.003	m
Near-cylinder size $l_{c,near}$	0.001	m
Extrusion layers (z)	1	—
Max aspect ratio	28.9	—
Max non-orthogonality	47.2	degrees
Max skewness	0.94	—

A.3 Boundary Conditions

Table 6 lists the boundary condition applied to each patch. Conditions are identical across both Reynolds numbers; only the inlet velocity magnitude, via U_{mean} in equation 9, differs between cases.

Table 6: Boundary condition specification for all patches.

Boundary	Patch type	Velocity (U)	Pressure (p)
Inlet ($x = 0$)	patch	Parabolic profile (equation 9)	zeroGradient
Outlet ($x = 2.2$)	patch	zeroGradient	fixedValue $= 0$
Top/bottom walls	wall	noSlip	zeroGradient
Cylinder surface	wall	noSlip	zeroGradient
Front/back faces	empty	empty (2-D)	empty (2-D)

A.4 OpenFOAM Solver Configuration

Both Reynolds number cases are solved in OpenFOAM v2512 using `icoFoam`, a transient solver for incompressible, laminar flow; no turbulence model is applied, which is appropriate for $Re < 200$. Pressure–velocity coupling is handled by the PISO algorithm, with the correction and scheme settings summarised in Table 7.

Table 7: Solver and numerical scheme configuration applied to both simulations.

Parameter	Value
Solver	`icoFoam` (transient laminar NS)
Time scheme	Euler (1st-order)
Divergence scheme	Gauss linearUpwind
Laplacian scheme	Gauss linear corrected
Pressure solver	GAMG + Gauss–Seidel
Velocity solver	smoothSolver + Gauss–Seidel
PISO correctors	3
Non-orthogonality correctors	2
Turbulence model	Laminar (none), appropriate for $Re < 200$
Fluid density ρ	1.0 kg/m^3
Kinematic viscosity ν	0.001 m^2/s

A.5 REYNOLDS NUMBER 20: STEADY FLOW

At $Re = 20$, obtained with a mean inlet velocity $U_{mean} = 0.20$ m/s (equation 8), the flow is laminar and converges to a steady, symmetric wake by approximately $t = 5$s. We simulate to $t = 10$s at $\Delta t = 0.001$s, well within the stability limit (max Courant number 0.58), and write the full flow field every 0.1s for 100 snapshots. Our converged drag coefficient of $C_d = 5.567$ matches the published ST reference value of 5.579 to within 0.2%, validating the simulation before use as training data (Table 8).

Table 8: Time integration and output parameters for the $Re = 20$ (steady) simulation.

Parameter	Value	Units / notes
Reynolds number Re	20	—
Peak inlet velocity U_m	0.30	m/s
Mean inlet velocity U_{mean}	0.20	m/s $= \frac{2}{3}U_m$
Time step Δt	0.001	s
Simulation end time	10.0	s
Write interval	0.1	s (every 100 steps)
Number of snapshots	100	—
Max Courant number	0.58	—
Flow regime	Steady-state	converges $\sim t = 5$s
C_d (converged)	5.567	ST ref. 5.579 (0.2% error)
Total data points	5,018,400	100 snaps $\times$ 50,184 cells

A.6 REYNOLDS NUMBER 100: UNSTEADY VORTEX SHEDDING

At $Re = 100$, obtained with a mean inlet velocity $U_{mean} = 1.00$ m/s, the flow is unsteady and sheds a periodic Kármán vortex street from approximately $t = 4$s onward. The higher velocity and finer near-wall dynamics require a smaller time step: we simulate to $t = 8$s, the duration specified by the ST benchmark, at $\Delta t = 0.0002$s (max Courant number 0.85), again writing the full flow field every 0.1s, for 80 snapshots. Our time-averaged drag coefficient over the periodic regime ($t \in [4, 8]$s) matches the ST reference to within 7.8%, while lift amplitude and Strouhal number both match to within 1% (Table 9). The larger drag discrepancy reflects the known sensitivity of integrated drag to near-wall mesh resolution at higher Reynolds number; it sets a practical upper bound on how closely a PINN trained on these fields can match the true ST solution, but does not affect the self-consistency of our evaluation framework.

Table 9: Time integration and output parameters for the $Re = 100$ (unsteady) simulation.

Parameter	Value	Units / notes
Reynolds number Re	100	—
Peak inlet velocity U_m	1.50	m/s
Mean inlet velocity U_{mean}	1.00	m/s $= \frac{2}{3}U_m$
Time step Δt	0.0002	s
Simulation end time	8.0	s (ST benchmark spec)
Write interval	0.1	s (every 500 steps)
Number of snapshots	80	—
Max Courant number	0.85	—
Flow regime	Unsteady (vortex shedding)	periodic from $\sim t = 4$s
Mean C_d	3.170	ST ref. 3.44 (7.8% error)
Max $\lvert C_l \rvert$	0.935	ST ref. 0.928 (0.75% error)
Strouhal number St	0.298	ST ref. 0.300 (0.7% error)
Vortex shedding period T	$\sim$0.336	s
Total data points	4,014,720	80 snaps $\times$ 50,184 cells

B Full Field Comparisons

Figure 4 in the main text shows a single-quantity snapshot across four configurations. Here we provide the complete picture for three representative configurations, the baseline, our best-performing pairing scaled to a deeper model, and the Fourier Features + SIREN failure case, showing the true field (OpenFOAM), the PINN prediction, and the absolute error for all three physical quantities (U_x, U_y, p) at $t = 5.0$s and $t = 5.4$s.

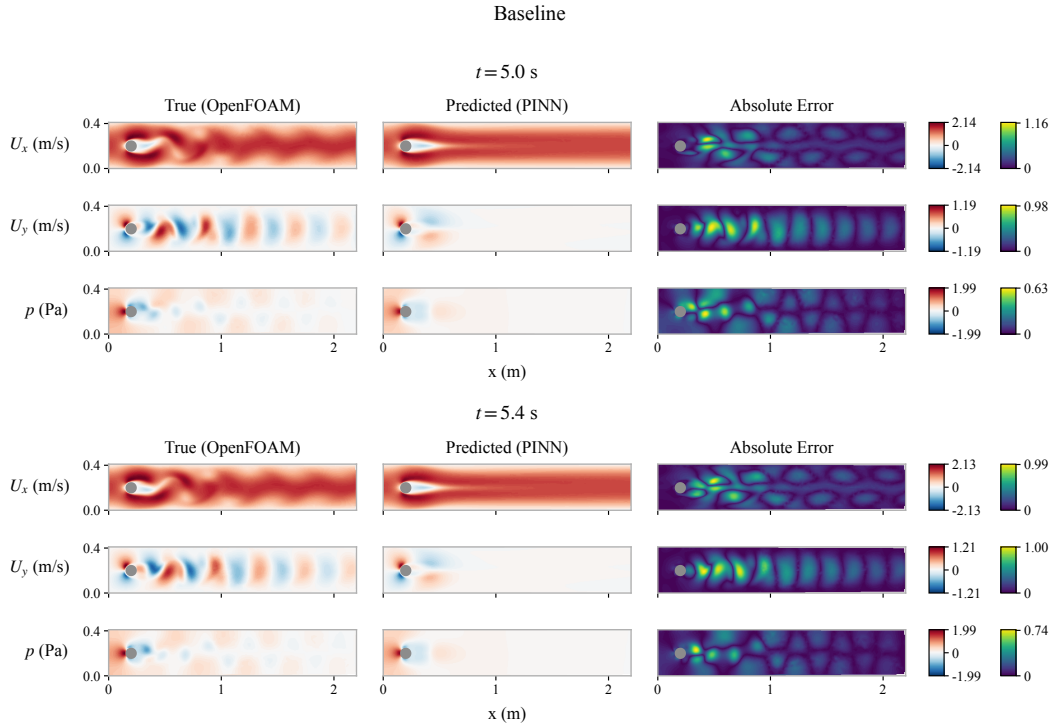
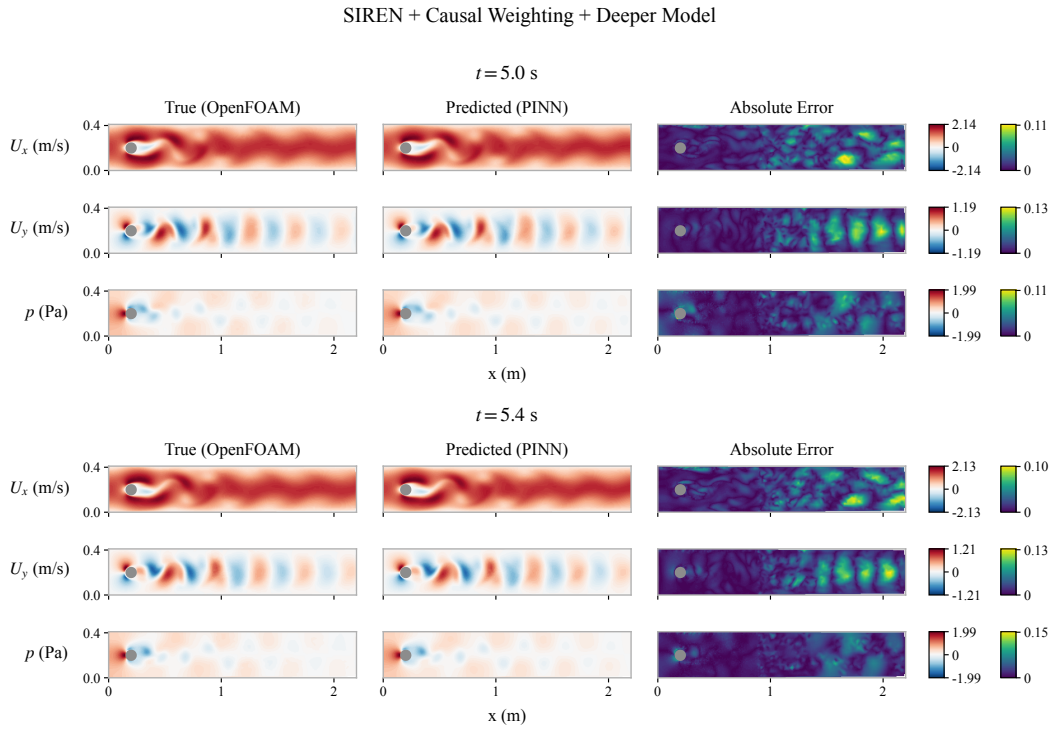
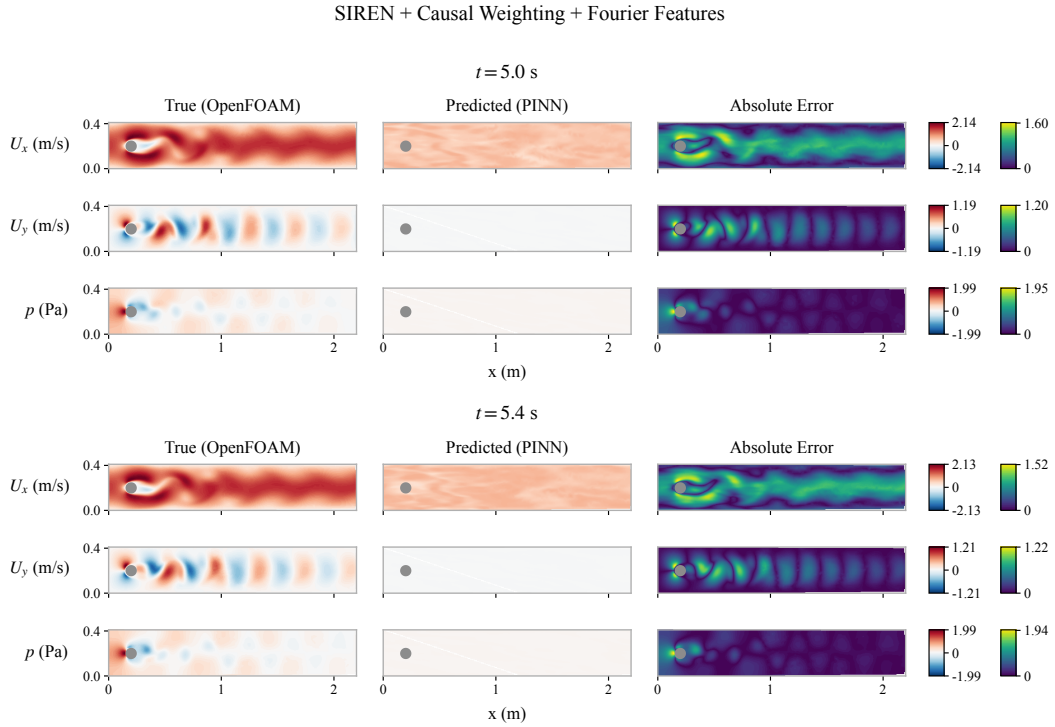


Figure 5: Full field comparison for the baseline configuration: true (OpenFOAM), predicted (PINN), and absolute error for U_x, U_y, and p at $t = 5.0$s and $t = 5.4$s. The baseline fails to reconstruct any wake oscillation, collapsing to a smoothed, near-steady near-field response.



(a) SIREN + Causal Weighting + Deeper Model. This configuration reconstructs the full vortex street with error concentrated in the far wake.



(b) SIREN + Causal Weighting + Fourier Features. The predicted fields collapse to a near-uniform, near-zero output across the entire domain, consistent with the output-layer collapse mechanism discussed in Section 5.2.